



Curso de Engenharia da Computação
Disciplina de Sistemas Operacionais
Prof. Bruno Silveira Neves

Especificação do Trabalho de Implementação

Organização das equipes: individual, duplas ou trios.

Cronograma de Entrega/Apresentação:

- Datas para apresentação do trabalho: conforme descrição das datas para os seminários, parcial e final, feitas no plano de ensino da disciplina.

Objetivo: Implementar em Java um algoritmo paralelo/concorrente para gerenciamento dinâmico de memória, usando recursos de mais baixo nível (semáforos ou monitores) para controlar o sincronismo entre threads.

Detalhamento:

Sobre o algoritmo:

- Implementar um gerador de requisições aleatórias que irá simular a criação de demandas para alocação de variáveis (espaços) na memória, tal como os programas reais fariam para solicitar a alocação das suas variáveis dinâmicas. Cada requisição deve especificar o tamanho (aleatório) da variável a ser alocada, devendo o simulador permitir que o usuário defina os valores mínimo e máximo para cada alocação de memória (exemplo: todas as requisições devem ficar na faixa entre 16 bytes (mínimo) e 1 KB (máximo)). O número de requisições a serem geradas durante os testes deve ser grande o suficiente para conferir uma boa carga de trabalho para o sistema de alocação de memória.
- O usuário do sistema deve ser capaz de configurar o tamanho (em KB) da heap¹ para funcionamento do sistema. Recomenda-se executar testes de desempenho com variados tamanhos de heap para avaliar o comportamento do sistema. A heap deve ser implementada como um vetor de inteiros (cada inteiro equivale a 4 bytes).
- Sempre que não houver espaço suficiente na heap para alocar uma variável, um algoritmo de liberação deverá ser chamado para remover da memória tantas variáveis quantas forem necessárias para permitir a entrada de novas variáveis na heap. Sempre que o algoritmo de liberação for chamado, ele deverá liberar um espaço equivalente a pelo menos 30 % do tamanho total da heap. O algoritmo de liberação deverá funcionar com uma lógica RANDOM, excluindo as variáveis da heap de forma aleatória.

¹ Área na memória usada para alocação das variáveis dinâmicas de um programa. Obs.: não confundir a heap a ser implementada no escopo deste trabalho com a heap utilizada pela máquina virtual Java para realizar a alocação das variáveis dinâmicas dos programas Java. Assim, o simulador deve conter sua própria heap desenvolvida pela equipe.

- A escolha do algoritmo para realizar a alocação de memória fica a livre arbítrio das equipes. Podem, portanto, ser usados algoritmos baseados tanto em políticas de alocação contígua como não contígua, cabendo às equipes distinguir as soluções mais complexas (e potencialmente mais eficientes) daquelas menos complexas. Optando por um algoritmo de alocação contígua, uma lógica de compactação da heap, será necessária para eliminar a fragmentação externa, quando necessário.
- Cada requisição deve ter um ID exclusivo e o valor deste ID deve ser escrito em todas as posições da heap ocupadas pela variável correspondente ao ID.
- O número de threads a ser utilizadas e sua estratégia de implementação para promover paralelismo e ganho de desempenho na execução do sistema, é também de livre arbítrio das equipes.
- Ao final da execução, o sistema deve apresentar um resumo das principais informações resultantes da execução do programa, tais como: número total de requisições atendidas (alocadas na heap), tamanho médio das variáveis alocadas, número total de variáveis removidas da heap, número de chamadas ao algoritmo de compactação da memória (quando a alocação contígua for usada) e tempo total de execução.

Sobre a avaliação:

- Comparar o desempenho do algoritmo paralelo/concorrente contra o desempenho de uma versão puramente sequencial do mesmo algoritmo. Selecionar configurações para teste que confirmem uma carga computacional alta o suficiente para permitir evidenciar diferenças de desempenho entre as versões paralela e sequencial.
- As características físicas (hardware) e lógicas (software) da máquina usada como plataforma para coleta de dados devem ser apresentadas.

Apresentação do trabalho (seminário parcial e final):

- Cada membro de uma equipe receberá, em ambos os seminários, cerca de 5 minutos para descrever sua contribuição para o trabalho. Em caso de desenvolvimento individual, a apresentação não deve ultrapassar 10 minutos. Posteriormente a cada apresentação, poderão ser feitas perguntas sobre o trabalho.
- É mandatório haver participação de todos os membros de cada equipe ao longo de todas as etapas de desenvolvimento do trabalho (planejamento, implementação e apresentação).

Material a ser entregue (em ambos os seminários, parcial e final):

- Slides da apresentação (usar formato .ppt e não .pptx).
- Código fonte (plenamente comentado).

Composição da nota do trabalho:

Nota trabalho = (Nota seminário parcial x 30%) + (Nota seminário final x 70%)

Critérios de Avaliação para o Seminário Parcial:

- Domínio teórico do tema Gerenciamento de Memória e das opções para realizar a alocação dinâmica de memória: 30%.
- Complexidade do algoritmo (sequencial) escolhido para gerenciar a memória, contemplando todos os aspectos descritos no detalhamento desta especificação: 35%.

- Implementação do algoritmo sequencial: 35%.

Critérios de Avaliação para o Seminário Final:

- Complexidade do projeto da versão paralela: 45%.
- Desenvolvimento de toda a implementação paralela: 35%.
- Desenvolvimento da análise comparativa contra a solução sequencial: 20%.